

MARTIN-LUTHER-UNIVERSITÄT
HALLE-WITTENBERG

Martin-Luther-Universität Halle-Wittenberg

Institut für Informatik der Martin-Luther-Universität Halle-Wittenberg

Hausarbeit Softwaretechnik

Entwicklung einer Software für das Spiel: Mühle

Name: Ahmad Wattar
Studiengang: Bioinformatik
Martikelnummer: 217234191
Semester: 7. Semester

Modulverantortlicher Prof.Dr. Wolf Zimmermann

23.02.2022

Inhaltsverzeichnis

I. Einleitung 4

II. Hauptteil 5

1. Prinzipien 5

2. Analysephase 6

2.1. Erstellen der Anwendungsfalldiagramme.....
2.1.1. Anwendungsfall Modellierung 7
2.1.2. Anwendungsfalldiagramm Spieler 8
2.1.3. Anwendungsfalldiagramm Turnierleiter 9
2.1.4. Anwendungsfall Administrator 10
2.1.5. Anwendungsfall Spielregeln 11
2.2. Anwendungsfälle spezialisieren 12
2.2.1. Paketdiagramm Modellierung 12
2.2.2. Klassendiagramm Spielverwaltung 13
2.2.3. Klassendiagramm Spielerverwaltung 14
2.2.4. Klassendiagramm Turnierverwaltung 15
2.3. Klassenverträge erstellen 16
2.4. Spezifikation nochmal überarbeiten 16
2.5. Pflichtenheft als Resultat der Analysephase 16

3. Entwurfphase 17

Grobentwurf 18

3.1. Paketdiagramm Entwurf 18
3.1.1. Paketdiagramm MuehleModel 19
3.1.2. Paketdiagramm MuehleView 20
3.1.3. Paketdiagramm MuehleController 21

Feinentwurf 22

3.2. FE Muehle 22
3.2.1. Klassendiagramm KDMuehleMVC 22
3.3. FE Spieler 23
3.3.1. Klassendiagramm SpielerObserver 23
3.3.2. Klassendiagramm SpielerController 24
3.3.3. Klassendiagramm SpielerModel 25
3.3.4. Klassendiagramm SpielerView 26
3.4. FE Spiel 27
3.4.1. Klassendiagramm SpielObserver 27

3.4.2. Klassendiagramm SpielController 27
3.4.3. Klassendiagramm SpielModel 28
3.4.4. Klassendiagramm SpielView 28

3.5. FE Turnier 29
3.5.1. Klassendiagramm TurnierObserver 29
3.5.2 Klassendiagramm TurnierController 29
3.5.3. Klassendiagramm TurnierModel 30
3.5.4. Klassendiagramm TurnierView 31

4. Implementierung **33**
4.1. Papyrus Codeerzeugung 33

III. Schluss **33**

IV. Literatur **33**

V. Abbildungsverzeichnis **34**

VI. Eigenständigkeitserklärung **35**

VII. Anahng

Teil I.
Einleitung

Die Softwaresysteme verbreiten sich in unserem Leben in allen unseren Lebensbereichen immer mehr. Egal ob bei einer Entwicklung von einfachen Spielen oder von eingebetteten Systemen wie z.B. Motoren, Airbags, Roboter und Flugzeuge, muss das System später bei der Anwendung keine Probleme verursachen, da es sonst zu unerwartetn katastrophen Ergebnisse gekommen könnte wie das Beispiel von den Flugzeugen in der Vorlesung, daher sind die Phasen der Softwareenticklung von einer großen Bedeutung. Das Thema meiner Hausarbeit ist ein Softwareprojekt, welches dient die erlerntet Strategien der Softwareentwicklung bestmöglich umzusetzen. Im Projekt werde ich das Spielbrett Mühle entwickeln, dabei werde ich auf die einzelnen Phasen der Softwareentwicklung eingehen, insbesondere die Analysephase, Entwurfphase und die Implementierung. Der Ausgangspunkt im Projekt ist das bereits gestellten Lastenheft, mit dem erstmal die Anwendungsfalldiagramme in der Analysephase erstellt werden, daraus lässt sich das Pflichtenheft erzeugen. Anhand diesen Softwatearchitekturen können dann Bausteine entworfen werden, die dann man implementieren kann. In der Implementierung wird der Quellcode geschrieben werden, der konsistent zu der Modellierung und zum Entwurf ist.

Teil II.
Hauptteil

1. Prinzipien

Die Grundlage für die Entwicklung des Softwareprojekts ist die Abstraktion und Konkre- tisierung. In der Abstraktion wird das vorhandene Lastenheft verwendet, um die Modellierung konstruieren zu können. Anschließend folgt die Konkretisierung, aus der das Pflichtenheft resultiert. Damit es eine Struktur in den einzelnen Prozessen gibt, wird das Prinzip der Hierarchisierung verwendet. Diese Struktur wird anschaulich durch die Pakete und die Anwendungsfälle und dann wird es in der Entwurf nochmal entschieden ob diese Struktur erhalten bleibet oder die noch geändert werden soll.

In jeder Stufe in der Softwaretechnik muss man grundlegend Prinzipien verfolgen: (Blazert Seite 29)

- Prinzip der Abstraktion: Durch die Abstraktion erfolgt die Trennung.
- Prinzip der Strukturierung: Die Strukturiereung dient zu einer besseren Verständlichkeit und Analysierbarkeit.
- Prinzip der Bindung und Kopplung: Minimierung der Kopplung zwischen den Systemkomponenten und Maximierung der Bindungen in den Systemkomponenten dienen der Einfachheit des ganzen Systems.
- Prinzip der Hierarchisierung: Innerhalb des Systems muss eine bestimmte Rangordnung gegeben sein. Dies erhöht neben der Strukturiereung die Verständlichkeit.
- Prinzip der Modularisierung: Dadurch wird eine weitgehende Kontextunabhängigkeit von der Umgebung erreicht.
- Geheimnisprinzip: Änderungen an den Datenstrukturen wirken sich nicht auf die Zugriffsmethoden aus. Dadurch wird ein lokale Änderbarkeit ermöglicht.
- Prinzip der Lokalität: Bei einer guten Lokalität befindet sich die Information innerhalb eines Systems genau an der Stelle, wo sie gebraucht wird. Das erleichtert die Verständlichkeit und somit Wartung und Einarbeitung.
- Prinzip der Verbalisierung: Hier werden die Konzepte in Worten erfasst. Das erleichtert die Verständlichkeit.

Aus diesen Prinzipien sieht man deutlich, dass manche Prinzipien komplementär sind, die anderen aber wiederum gegenseitig. Während der Modellierungsphase soll entschieden werden, welche Prinzipien im Fall eines Konflikts wichtiger sind. Dies wird im Pflichtenhefft aufgeführt. Aßerdem soll in der Modellierungsphase die Struktur sowie die Hirarchierung deutlich gemacht werden.
In der Analysephase sollten jedoch nach dem Gehimsprinzip keine Implementierungs- und Entwurfdetails vorkommen (Vorlesung, Kapital 3.1)

2. Analysephase

Da das Lastenheft bereits vorhanden ist, entfällt die Vorstudie und es wird direkt mit der Entwicklung eines objektorientiereten Modells (OOA-Modell) angefangen. Der Ausgangspunkt ist das Lastenheft und es wird ein Plan aus 4 Schritte verfolgt (Vorlesung, Kapital 3.4. Seite: 292)

1. Erstellen der Anwendungsfalldiagramme: ggf. Unterteilung in Subsysteme
2. Anwendungsfälle spezialisieren: dabei werden Klassendiagramme erstellt und die Unterteilung in Subssystemen kann noch erweitert werden, falles es notwendig ist.
3. Klassenverträge erstellen: dabei werden die Klassendiagramme erweitert.
4. Spezifikation nochmal überarbeiten: Wiederholung der Schritte 1-4

Anmerkung: Wenn ein Vertrag auf einen Diagramm nicht spezifiziert wurde, dann geht man davon aus, dass die entsprechende Klasse eine triviale Invariante true besitzt.

2.1. Erstellen der Anwendungsfalldiagramme

2.1.1. Anwendungsfall Modellierung

Das folgende Anwendungfalldiagramm, was im Paket Spielverwaltung liegt, zeigt wie der Spieler mit dem System integrieren kann. Es muss ihm möglich sein ein Spiel zu spielen. Das wird jetzt erstmal sehr grob vorgestellt und wird später im Laufe der Hausarbeit spezifiziert. (LFA1.10)

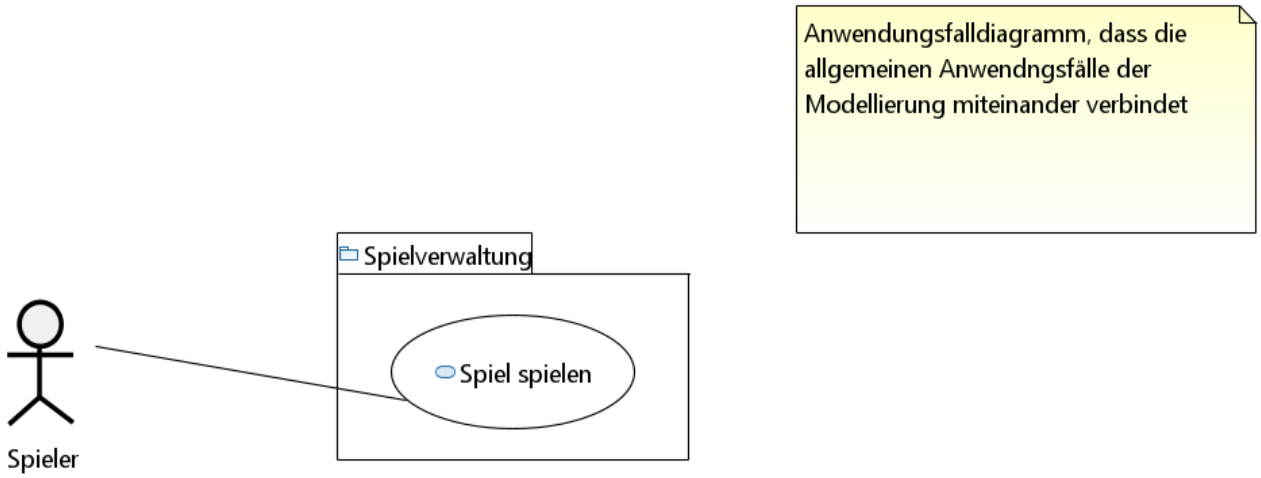


Abbildung 1: AFModellierung

2.1.2. Anwendungsfalldiagram Spieler

Das folgende Diagramm beschreibt was der Spieler machen kann. Durch die use-cases Spiel spielen, zuzweit spielen werden wird die LFA1.10 erfüllt.
Durch das Use-case Turnier spielen sird die LFA1.20 erfüllt und durch die Use-cases im Paket Spielerverwaltung wird die LFA1.40 erfüllt.

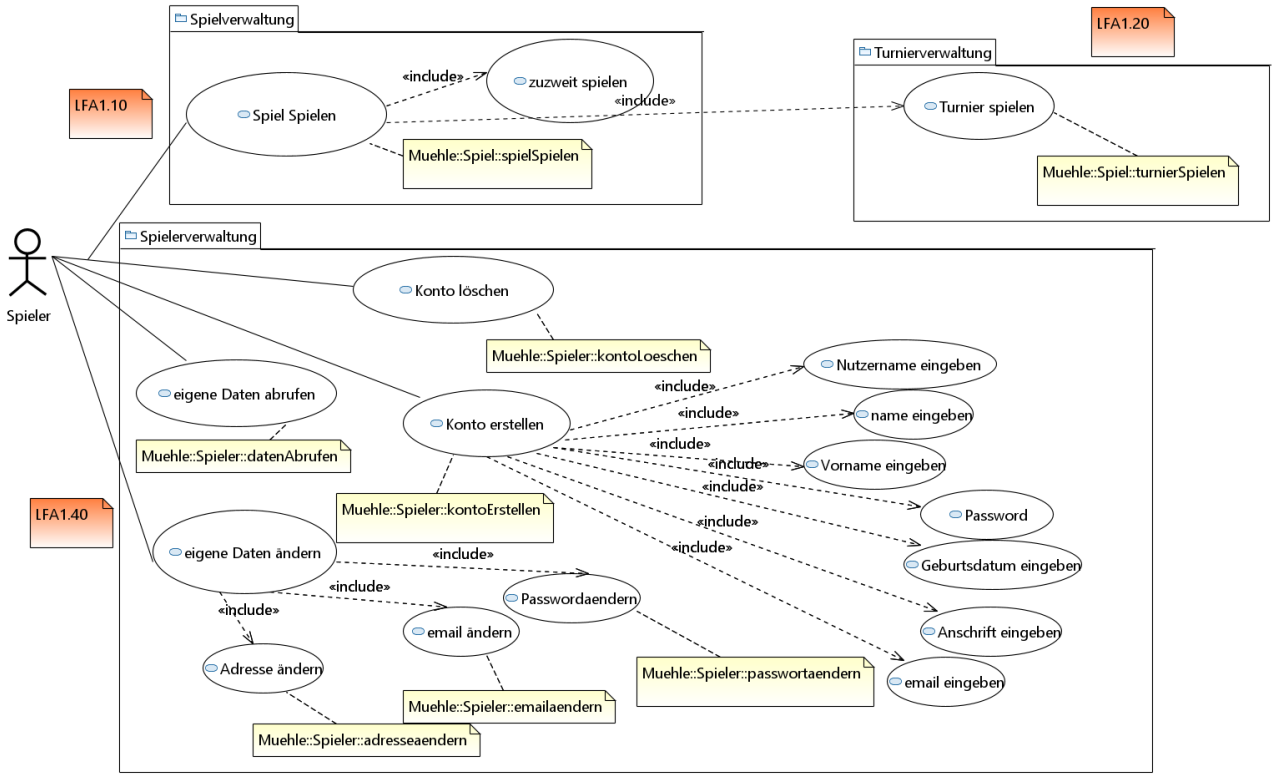


Abbildung 2: AFSpieler

2.1.3. Anwendungsfalldiagramm Turnierleiter

Das nächste Diagramm spezifiziert den Anwendungsfall Turnierleiter und wird weiter unterteilt. Mit diesem Diagramm wird die LF1.20 realisiert.

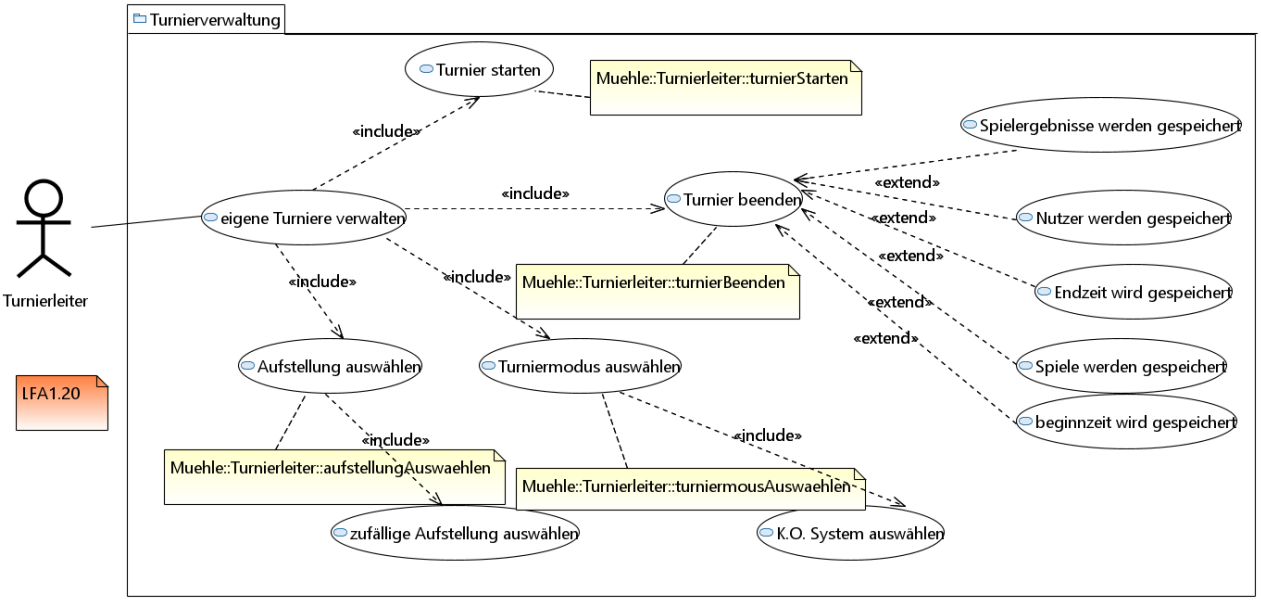


Abbildung 3: AFTurnierleiter

2.1.4. Anwendungsfall Administrator

Das folgende Anwendungsfalldiagramm beschreibt, was der Administrator machen kann. Durch die Use-cases im Paket Spielverwaltung wird die LFA1.10 realisiert.
Durch die anderen Use-cases im zweiten Paket „Spielerverwaltung“ wird die Punkt LFA1.40. erfüllt.

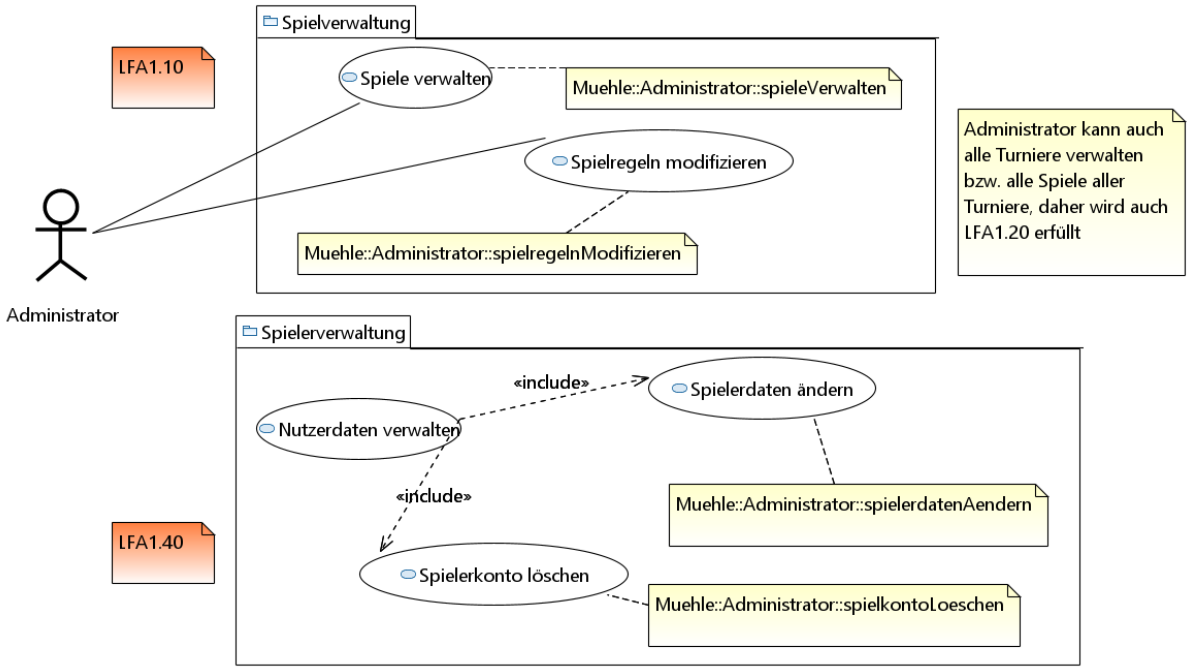


Abbildung 4: AFAdministrator

2.1.5. Anwendungsfall Spielregeln

Das folgende Anwengunsfalldiagramm beschreibt die möglichen bzw. erlaubten Spielzüge im Spiel anhand der Spielphase. Dadurch wird auch die LFA 1.30 realisiert.

Diese Anwendungsfalldiagramme sind wie in der Übung besprochen disjunkt, damit keine Überschneidung mit der Konsistenz geben würde.

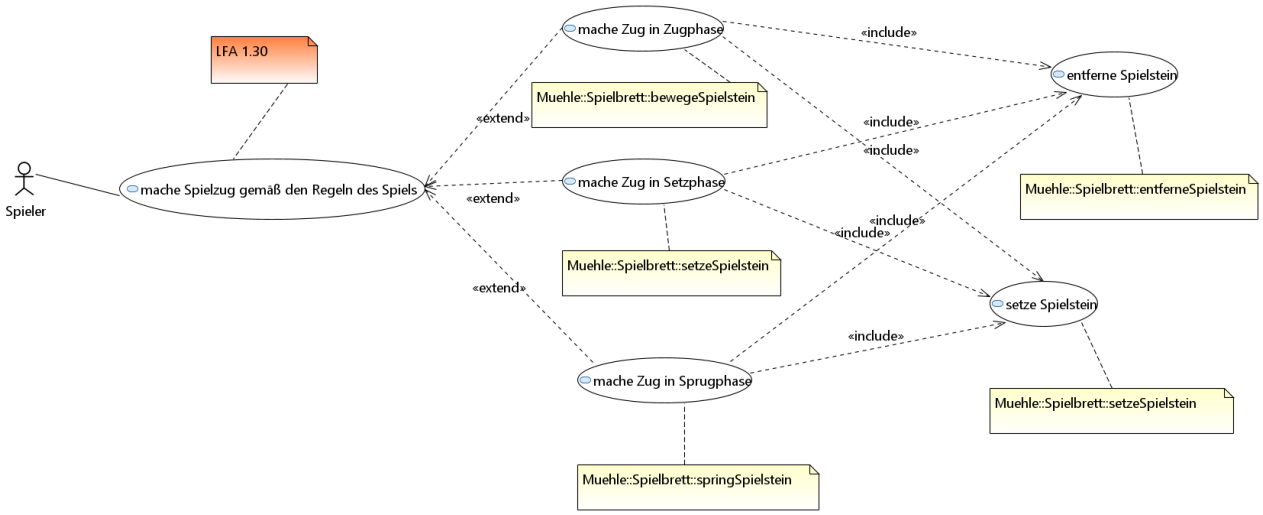


Abbildung 5: AFSpielregeln

2.2. Anwendungsfälle spezialisieren

2.2.1. Paketdiagramm Modellierung

Die folgenden zwei Pakete „Spielerverwaltung“ und „Muehle“ sind für eine spätere Entwicklung Des Projekts relevant.
Das Paket Spielerverwaltung ist dafür da, wenn später andere Brettspiele dazu kommen und das andere Paket Muehle ist für die Unterteilung des Spieles in den Unterpaketen Spielverwaltung, Spielerverwaltung und Turnierverwaltung.

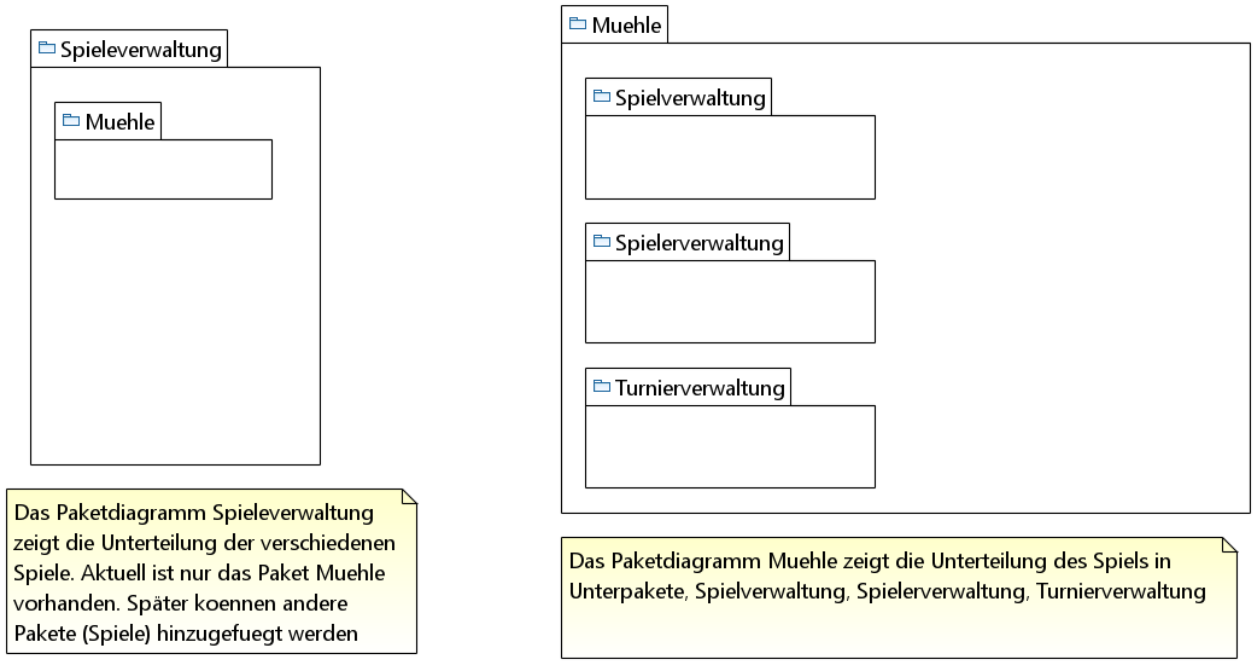


Abbildung 6: PDModellierung

2.2.2. Klassendiagramm Spielverwaltung

Die Anwendungsfalldiagrammen sollen im Laufe der Analysephase mit Klassendiagrammen erweitert werden.
Alle private Attribute haben Setter und Getter Methoden, bis auf das Spielscore in der Klasse Spielerverwaltung, was nur eine Get-Methode hat, um Manipulationen zu vermeiden.
Diese Setter und Getter sind in den Diagrammen nicht vorhanden (wie in der Übung besprochen) damit die Diagramme anschlaulicher sind.
Im Klassendiagramm KDSpielverwaltung was im Paket Spielverwaltung liegt, sind die Klassen Spiel, Spielbrett, Knoten und Spielsteine zu sehen.
LFA1.10 ist erfüllt.

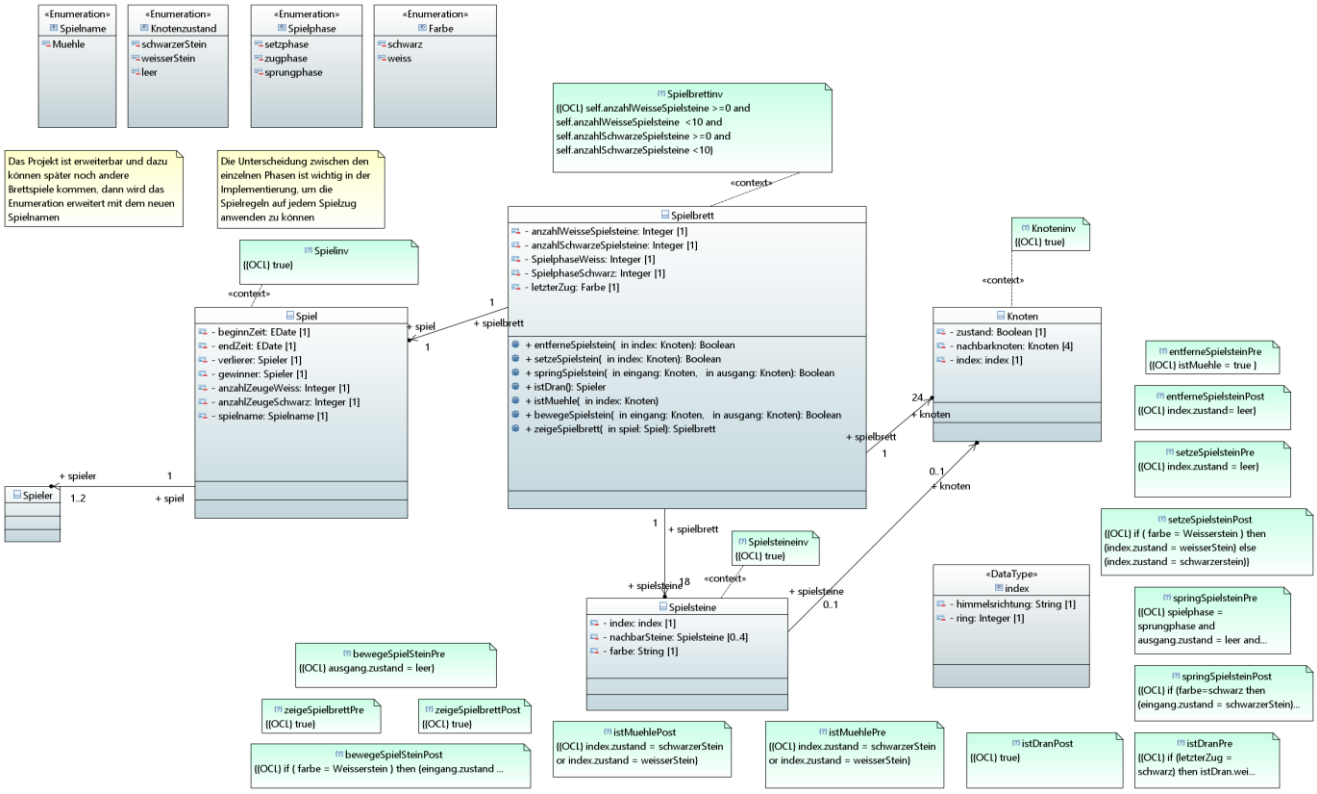


Abbildung 7: KD Spielverwaltung

2.2.3. Klassendiagramm Spielerverwaltung

Im Klassendiagramm KDSpielerverwaltung, was im Paket Spielerverwaltung zu finden ist, sind hauptsächlich die Klassen Spieler, Turnierleiter, Administrator.
Dadurch ist die LFA1.40 erfüllt.

Hier sind die verschiedenen Methoden in der Klasse Spieler zu sehen. Ein Spieler kann sich regestriren und ein Konto erstellen, in dem er seine Daten eingibt wie Name, Geburtsdatum usw. Ein Spieler kann außerdem seine Daten ändern und das Konto löschen. Somit ist LFA1.40. erfüllt

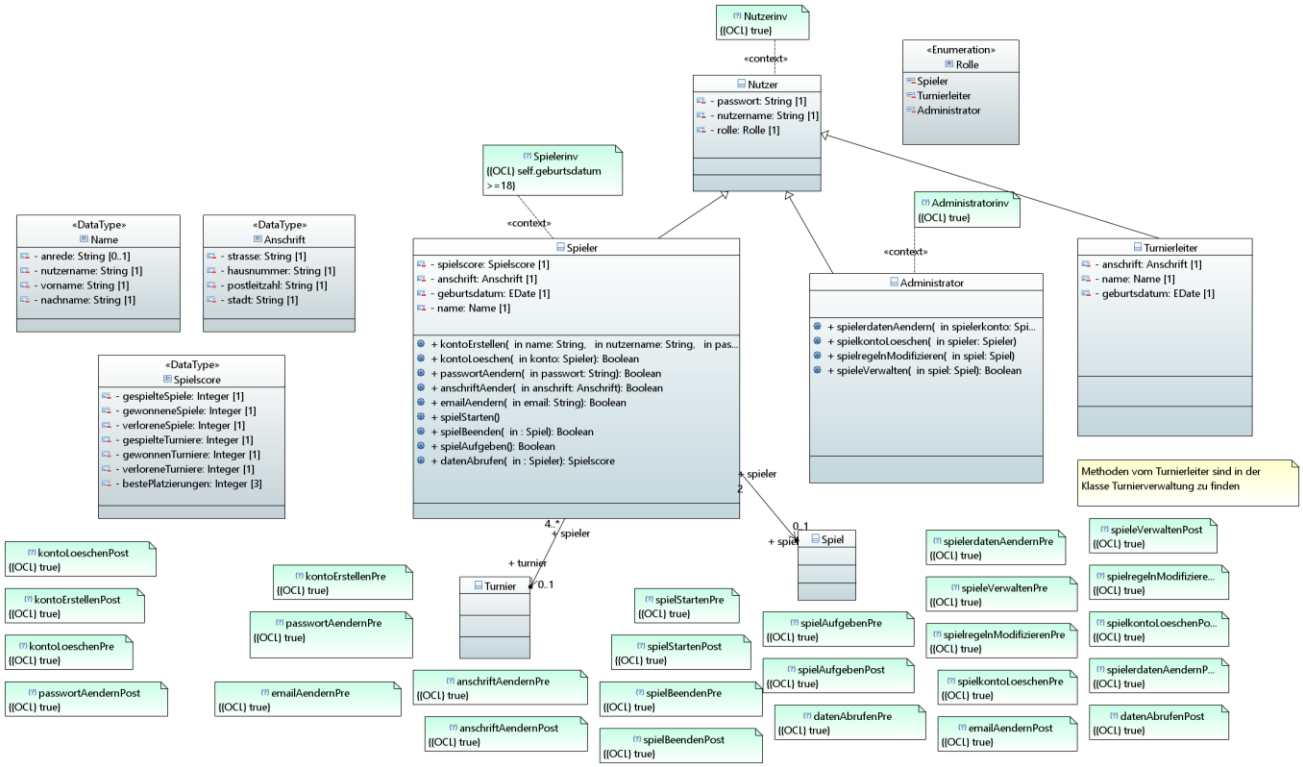


Abbildung 8: KD Spielerverwaltung

2.2.4. Klassendiagramm Turnierverwaltung

Durch das folgende Klassendiagramm, was im Paket Turnerverwaltung zu finden ist, wird die LFA1.20 erfüllt.
Die beiden Enumeration sind wichtig, um später sowohl das Turniermodus als auch die Aufstellungsart erweitern zu können.

Es wurde die zwei Enumeration zugeführt, da sie später laut Auftragsgeber erweitert werden. Es steht also später nicht nur KO-System und automatisch (zufüllig) sondern was noch dazu.

In der Klasse Turnierleiter sind die verschiedenen Methoden zu sehen, wie z.b. Turnierstraten, Turnierbeenden usw., daher ist die LFA1.20. realisiert.

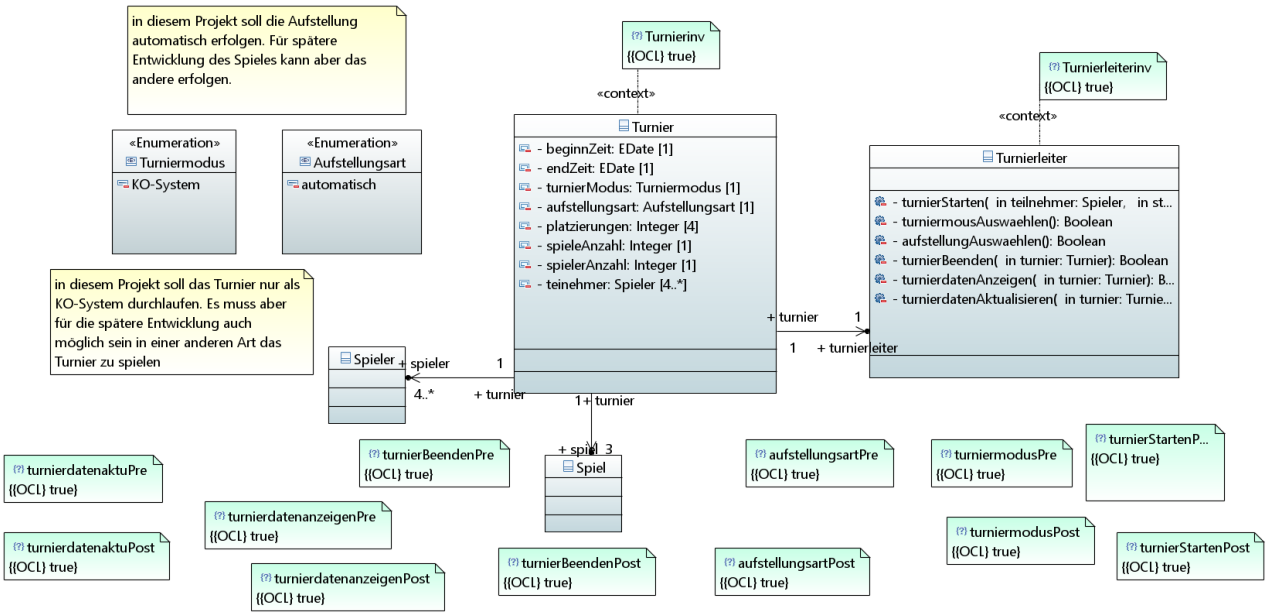


Abbildung 9: KD Turnierverwaltung

2.3. Klassenverträge erstellen

Die Klasseninvariant Spielbrettinv beschreibt, dass es auf dem Spielbrett entweder null weisse Steine oder maximal 9 stehen können und geneuso das gleiche für die Farbe schwarz.
In der Klasse Spieler ist die Invariante, dass der Spieler mindestens 18 sein muss, um spielen zu können. (Anforderung vom Auftragsgeber).
Bei den anderen Klassen, wo es true ist, gibt es keine Anforderungen.

2.4. Spezifikation nochmal überarbeiten

Sowohl die funktionalen Anforderungen, die im Lastenheft stehen, als auch die Wünsche Vom Auftragsgeber wurden vollständig umgesetzt, sodass das Modell keinen Wiederspruch besitzt und die Konsistenz des Modells weiter gewährleistet ist. Die Qualität des Modells ist also sicher, sodass es nicht notwendig die Spezifikation zu bearbeiten.

2.5. Pflichtenheft als Resultat der Analysephase

Die Analysephase wurde mit der Fertigstellung des Pflichtenhefts abgeschlossen. Die Anforderungen, die im Lastenheft aufgeführt wurde, wurden im Pflichtenheft analysiert.
Das Pflichtenheft wird am Ende der ganzen Dokumentation als Anhang aufgeführt.

3. Entwurf

Das Vorgehen vom Entwurf orientiert sich an den Folien der Vorlseung (Kapital 4, Seite: 367) und am Buch von Balzert (Seiten 483- 485)

1. Anwendungsart festlegen:
Die Anwendungsartt ist eine Mehrplatzanwendung, da es eine Datenfähigkeit wichtig ist, denn es gibt verschiedene Akturen bzw. Rollen (Spieler, Turnierleiter, Administrator).

2. Nichtfunktionale Anforderungen prioriesieren:
Erweiterbarkeit und Änderbarkeit (s. Pflichtenheft)

3. Verteilungsart: Nicht verteilt

4. Grobentwurf

Die Modellierung und der Entwurf wurden mit der der Papyrus Version papyrus-2020-06- und eclipse 2020-06 modelliert, da die andere Version 2020-12 zu vielen Problemen und Fehlermeldungen geführt hat. Die Fehlermedlungen konnte ich am Ende beheben und habe den Hinweis von der Übung gentzt, dass ich einige OCL den Typ „Unlimited Natural“ gegeben habe.
Es wurde das MVC-Muster gewählt, da es besonders den Qualtiätsanforderungen entspricht. MVC steht für die 3 Subsysteme Model-View-Controller. MVC ist ein objektbasiertes Verhaltenmuster und wird sowohl als Entwurfmuster als auch als Architekturmuster verwendet.
Das Model übernimmt die Datenhaltung und die dazugehörige facliche Logik, also die Verwaltung. Der Controller kann der Zustand eines Objekts ändern und das Subsystem View ist die Bentzungsoberfläche (nach Blazert Seite 62-63)
In diesem Projekt wird keine GUI gebraucht, sondern es läuft alles auf der Konsole, daher ist View wichtig für die Ausgabe für den Nutzer, wie z.b Spielbrett.
Durch diese Trennung in den Subsystemen ist die LNA 1000: Die Spielerdaten sollen geschützt sein, erfüllt.
Modell-View-Controller unterstützt die Wartbarkeit, da das Modell ist so unabhängig vom View gewartet und verändert werden kann.

Vorteile einer MVC sind: (nach Blazert Seite 68)
- Gute Erweiterbarkeit der Funktionalität einer Anwendung
- Unterstütung der Wartbarkeit der Anwendung, da durch Änderungen im Umfeld hervorgerufene Wartungsarbeiten im allgemeinen nur Teilsysteme aus einer Kategorie betreffen, daher ist LNA2000 auch realisiert

Also es unterstüzt die nichtfunktionalen Anforderungen Wartbarkeit, Weiterentwickelbarkeit, Portabilität und Wiederverwendbarkeit, was am besten zur Entwicklung des Spieles geeignet ist, denn das Projekt wird später noch entwickelt und erweitert.

Grobentwurf

Hier wird die globale Softwarearchitektur festgelegt.

3.1. Paketdiagramm Entwurf

Das Paketdiagramm PDEntwurf zeigt die Architektur des Entwurfs. Es ist untergliedert die Subsysteme in MuehleModel, MuehleView und MuehleController.

Mit dem Paket Mühle und die 3 Subsysteme erfollgt die Modellierung des Projekts

Die Beschreibung von einzelnen Pakten ist in den Kommentaren.

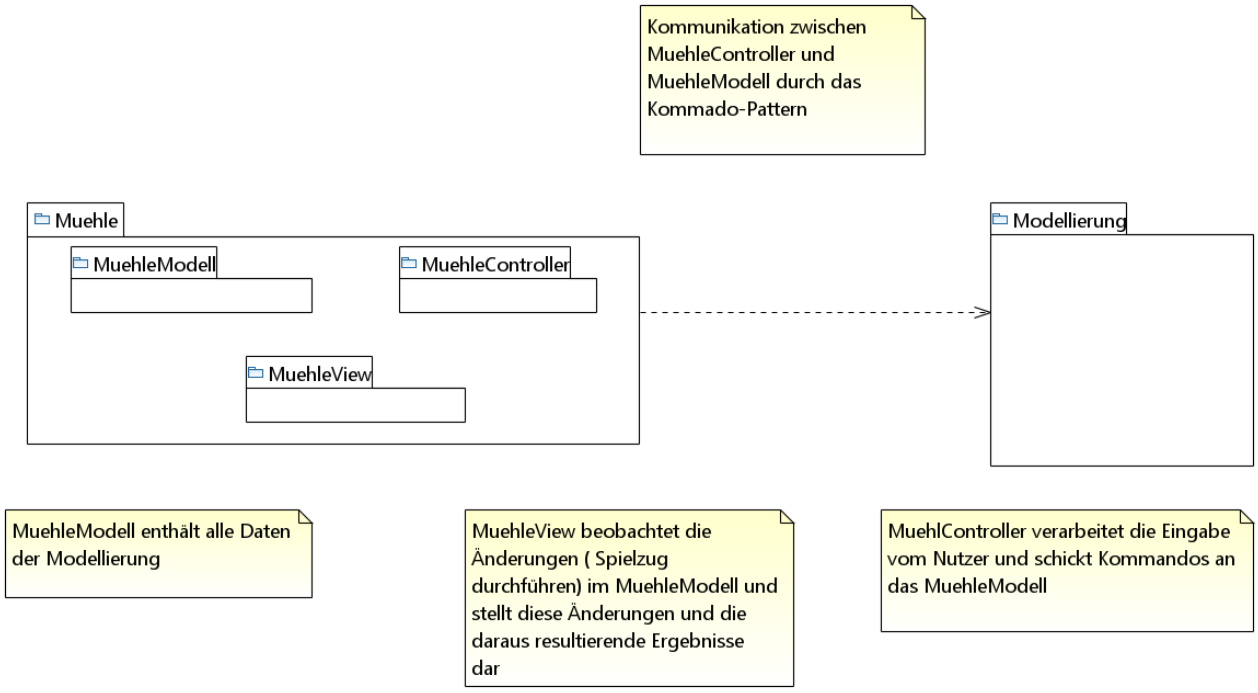


Abbildung 10: PDEntwurf

3.1.1 Paketdiagramm MuehleModel

Im Paketdiagramm MuehleModel sind die Pakete SpielerModell, SpieleModell und TurnierModell zu sehen.

Ich habe mich dafür entschieden, dass Spiel, Spieler, Turnier jeweils ein Modell, ein View und ein Controller haben, damit eine spätere Entwicklung des Projekts erleichtert wird. Diese Entscheidung dieses Entwurfes trägt auch der Trennung von Zuständigkeiten bei.

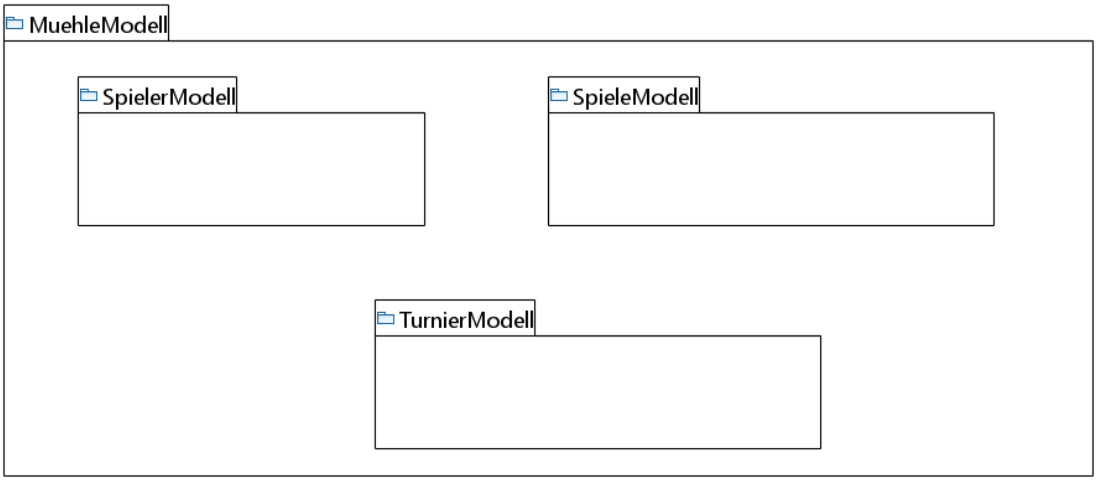


Abbildung 11: PDMuehleModel

3.1.2. Paketdiagramm MuehleView

Das Paket MuehleView wird wiederum in 3 weitere Pakete unterteilt die Pakete SpielerView, SpieleView und TurnierView zu sehen.

Die entsprechende Inhalt der Unterpakete und Bindungen zwischen den Unterpaketen des Subsystems werden in den Klassendiagrammen spezifiziert.

MuehleView registriert Änderungen in MuehleModel und zeigt diese auf der Konsole.

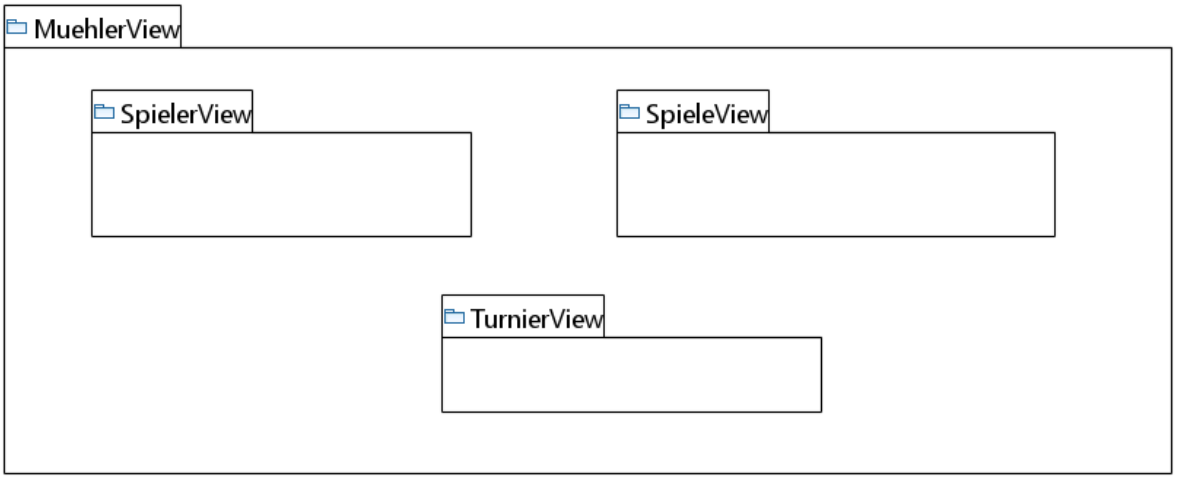


Abbildung 12: PDMuehleView

3.1.3. Paketdiagramm MuehleController

Im Paketdiagramm MuehleController sind die Pakete SpielerController, SpieleController und TurnierController zu sehen.

Diese Entscheidung dieses Entwurfes trägt der Trennung von Zuständigkeiten bei.

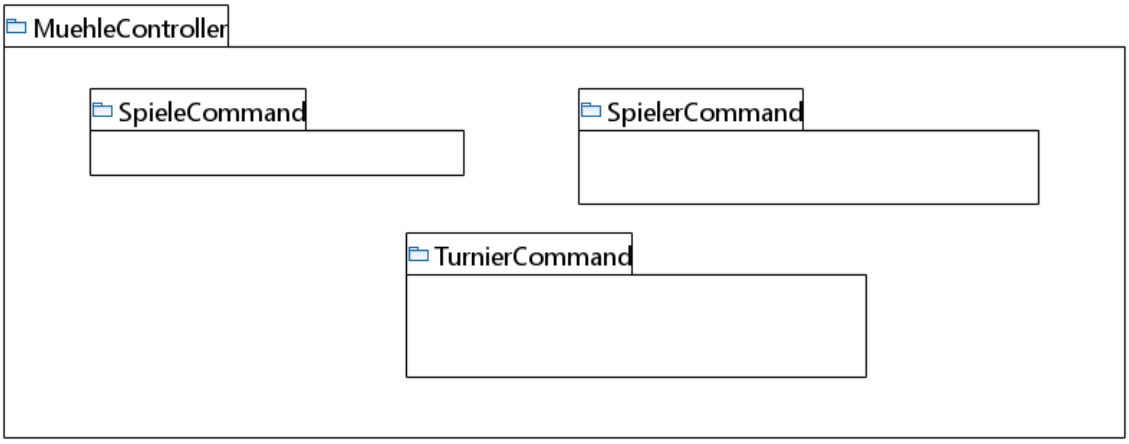


Abbildung 13: PDMuehleController

6. Softwaretechnische Infrastruktur:

Papyrus, Java

8. Kontext:

Spielmühle mit verschiedenen Rollen

9. Feinentwurf

Hier werden die einzelnen Subsysteme und Komponenten im Detail entworfen.

10. Evaluation der Architektur

Bevor mit der Implementierung der Anwendung begonnen wird, sollte geprüft werden, ob die entworfene Softwarearchitektur sich zur Erfüllung der nichtfunktionalen Anforderung tatsächlich eignet.

Feinentwurf

Hier werden die einzelnen Subsysteme und Komponenten im Detail entworfen, unter der Berücksichtigung der funktionalen Anforderungen und aller zuvor gemachten Entscheidungen und Einflussfaktoren.

3.2. FE Muehle

3.2.1. Klassendiagramm KDMuehleMVC (allgemein)

Hier sind die Schnittstellen vom jedem Paket (Subsystem), um mit den anderen Subsystem zu kommunizieren.
Das MVC ist eine Erweiterung des Observer Patterns.

ModelObserver beobachtet das Modelsubject, ob sich das Modell geändert hat. Ist das der Fall wird diese Änderung mittels ModelSubject am Observer übermittelt, damit diese Änderungen Ausgegeben werden kann.

ControllerSubject im Paket MuehleController nimmt die Eingaben und Befehle und gibt diese Am ControllerObserver weiter, um Änderungen im Model vorzunehmen.

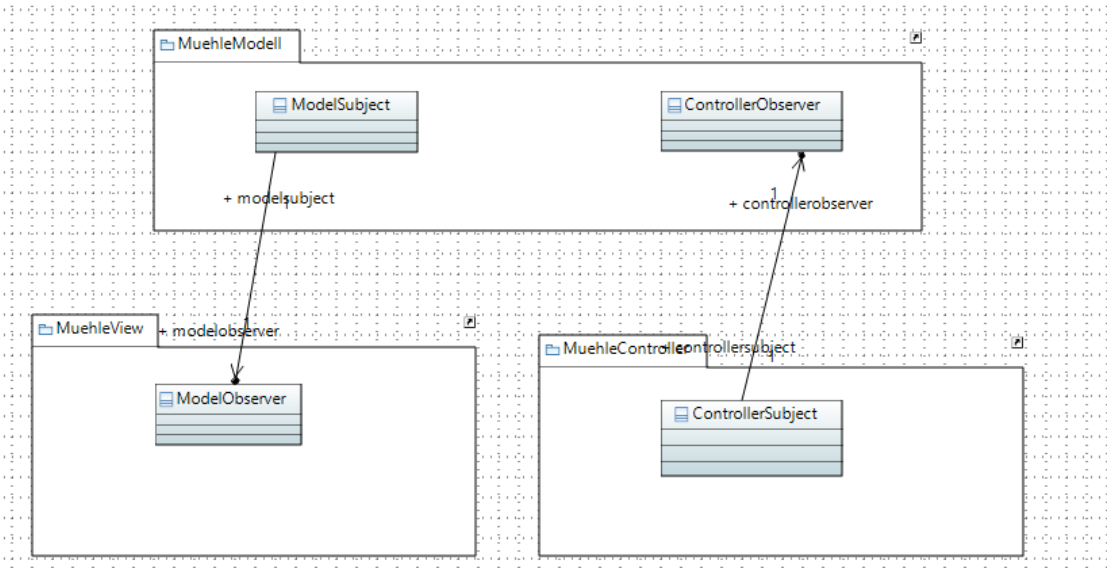


Abbildung 14: KDMuehleMVC

3.3.1. Klassendiagramm KDSpielerObserver

Die Klassen SpielerObserver, Subject sind abstrakt. Die Klasse Subject ist dafür da, um eine Menge von Observern zu speichern. Es können außerdem mit der Methode addObserver weitere Observer hinzugegeben werden.
Die Methode notify ist dafür da, um Benachrichtigungen an allen Observern, die gespeichert sind schicken zu können mit irgendeinem bestimmten Event und einem bestimmten Datentyp.
Evente sind im Enumeration zu sehen und die Datentypen im Datatype SpielerData

SpielerObserver wird mit einem Event benachrichtigt und Subject löst diese Benachrichtigung aus und merkt sich welche Observer, die benachrichtigt

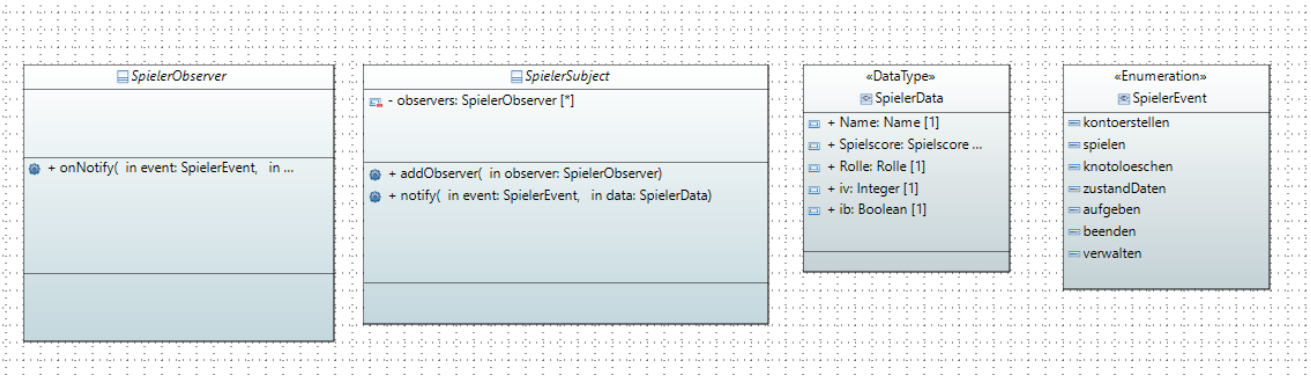


Abbildung 15: KDSpielerObserver

3.3.2. Klassendiagramm KDSpielerController

Spielerinterface mit der Main-Funktion. Hier gehen die Befehle ein und werden dann am ControllerSubject weitergegeben.
Es wird auch in der Klasse ControllerSubject, welche Befehle werden aufgenommen und weitergeleitet, um darauf zu reagieren.
In der Klasse ControllerSubject ist wie oben beschrieben die Methode notify zu sehen.

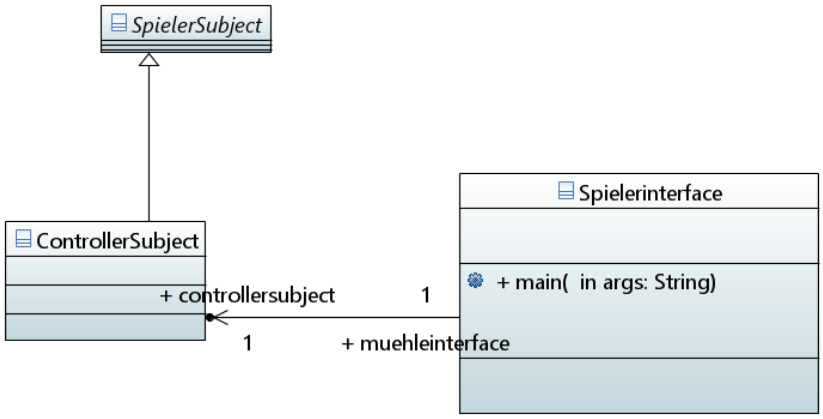


Abbildung 16: KDSpielerController

3.3.3. Klassendiagramm KDSpielerModel(Entwurf)

ControllerObserver wartet auf die Befehle und steurt mit deisen Befehlen im Prinzip die Klasse Spieler anhand Was für ein Befehl gegeben wurde.
Alle Observer erben vom abstrakten Observer. Hier im Fall ist *SpielerObserver*
Alle Subjects erben vom abstrakten Subjects. Hier im Fall ist *SpielerSubject*
Durch diese Vererbung wird die Austauschbarkeit ermöglicht.

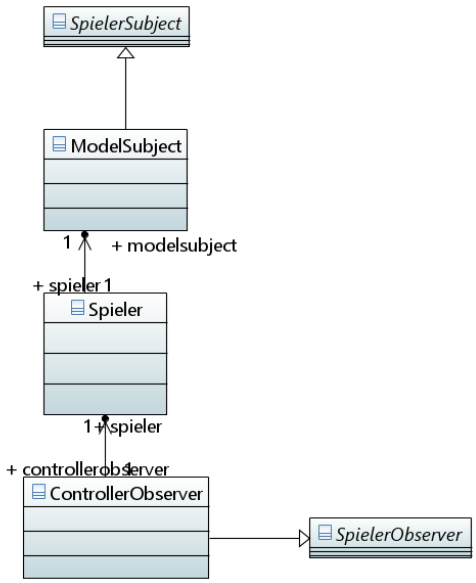


Abbildung 17: KDSpielerModell

3.3.4. Klassendiagramm KDSpielerView

SpielerAusgabe: Alle Methoden sind statisch und sind dafür da, damit was im Abhängigkeit von Spielerdaten für den Nutzer ausgegeben werden kann

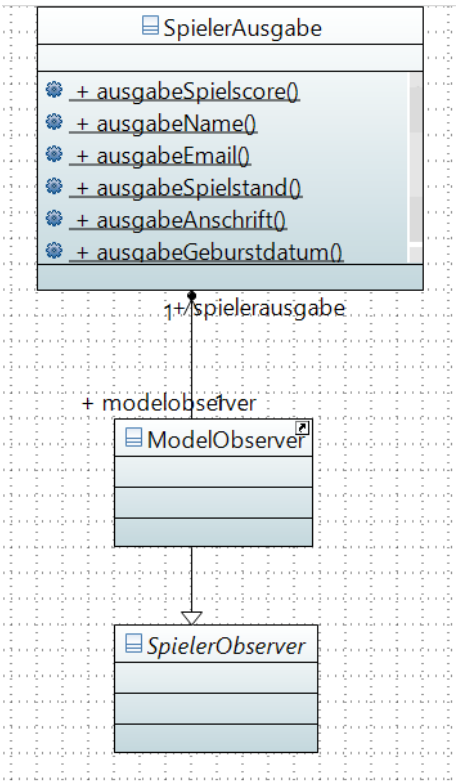


Abbildung 18: KDSpielerView

3.4.1. Klassendiagramm KDSpielObserver

Die Klassen SpielObserver, Subject sind abstrakt. Die Klasse Subject ist dafür da, um eine Menge von Observern zu speichern. Es können außerdem mit der Methode addObserver weitere Observer hinzugegeben werden.
Die Methode notify ist dafür da, um Benachrichtigungen an allen Observern, die gespeichert sind schicken zu können mit irgendeinem bestimmten Event und einem bestimmten Datentyp.
Evente sind im Enumeration zu sehen und die Datentypen im Datatype SpielData

SpielObserver wird mit einem Event benachrichtigt und Subject löst diese Benachrichtigung aus und merkt sich welche Observer, die benachrichtigt

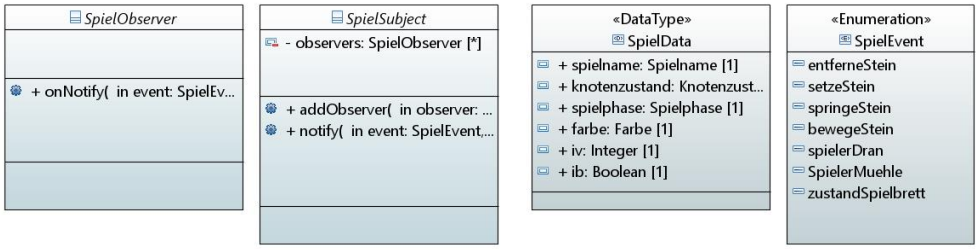


Abbildung 19: KDSpielObserver

3.4.2. Klassendiagramm KDSpielController

Spielinterface mit der Main-Funktion. Hier gehen die Befehle ein und werden dann am ControllerSubject weitergegeben.
Es wird auch in der Klasse ControllerSubject, welche Befehle werden aufgenommen und weitergeleitet, um darauf zu reagieren.
In der Klasse ControllerSubject ist wie oben beschrieben die Methode notify zu sehen.

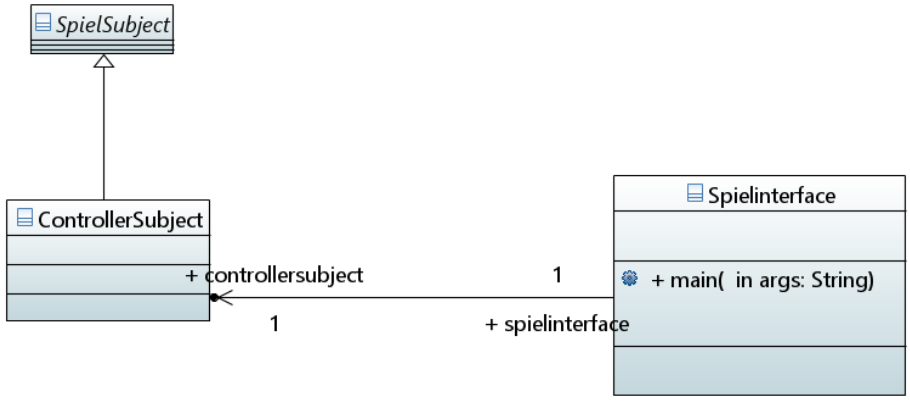


Abbildung 20: KDMuehleController

3.4.3. Klassendiagramm KDSpielModel(Entwurf)

ControllerObserver wartet auf die Befehle und steurt mit deisen Befehlen im Prinzip den Klassen Spiel und Spielbrett anhand was für ein Befehl gegeben wurde.
Alle Observer erben vom abstrakten Observer. Hier im Fall ist *SpielObserver*
Alle Subjects erben vom abstrakten Subjects. Hier im Fall ist *SpielSubject*
Durch diese Vererbung wird die Austauschbarkeit ermöglicht.

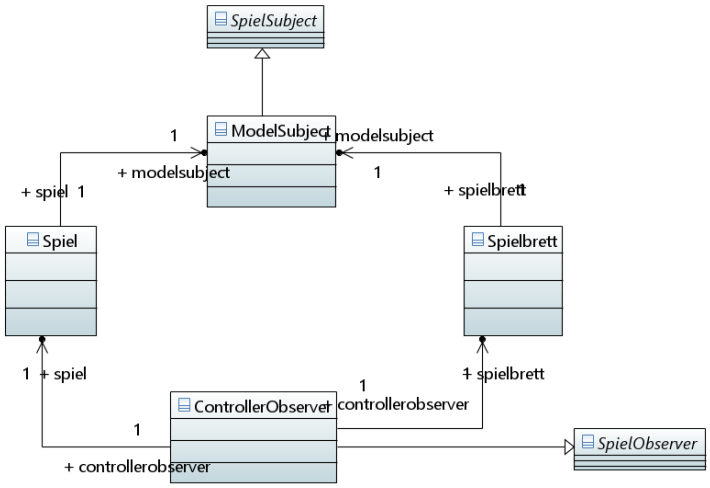


Abbildung 21: KDSpielModel

3.4.4. Klassendiagramm KDSpielView

SpielerAusgabe: Alle Methoden sind statisch und sind dafür da, damit was in Abhängigkeit Spieldaten für den Nutzer ausgegeben werden kann.

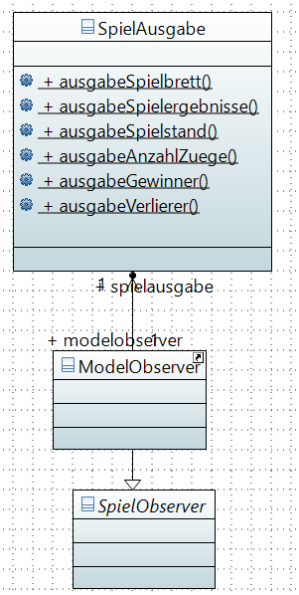


Abbildung 22: KDSpielView

3.5.1. Klassendiagramm KDTurnierObserver

Die Klassen TurnierObserver, Subject sind abstrakt. Die Klasse Subject ist dafür da, um eine Menge von Observern zu speichern. Es können außerdem mit der Methode addObserver weitere Observer hinzugegeben werden.
Die Methode notify ist dafür da, um Benachrichtigungen an allen Observern, die gespeichert sind schicken zu können mit irgendeinem bestimmten Event und einem bestimmten Datentyp.
Evente sind im Enumeration zu sehen und die Datentypen im Datatype TurnierData

TurnierObserver wird mit einem Event benachrichtigt und Subject löst diese Benachrichtigung aus und merkt sich welche Observer, die benachrichtigt

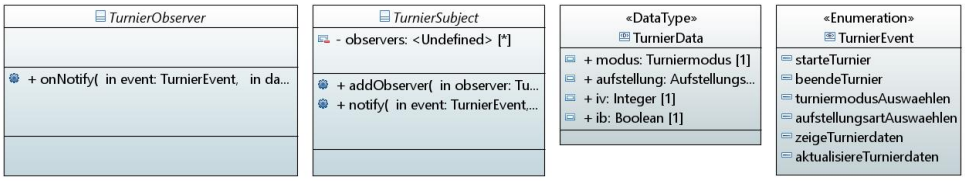


Abbildung 23: KDTurnierObserver

3.5.2. Klassendiagramm KDTurnierController

in dieser Main-Funktion gehen die Befehle ein und sie werden dann am ControllerSubject weitergegeben Die weitergegeben Befehle vom Turniernterface werden dann in der notify-Funktion bearbeitet im Sinne von (welche Befehle Sinn machen und angenommen werden und was dann aufgrund dieser Befehle gemacht werden soll)

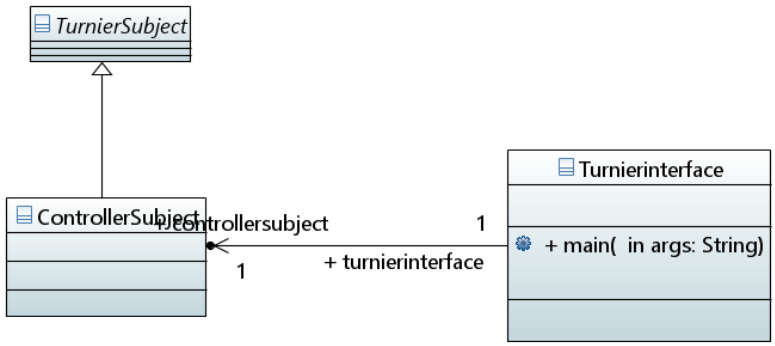


Abbildung 24: KDTurnierController

3.5.3. Klassendiagramm KDTurnierObserver

ControllerObserver wartet auf die Befehle und steurt mit deisen Befehlen im Prinzip den Klassen Turnier und Turnierleiter anhand was für ein Befehl gegeben wurde.
Alle Observer erben vom abstrakten Observer. Hier im Fall ist *TurnierObserver*
Alle Subjects erben vom abstrakten Subjects. Hier im Fall ist *TurnierSubject*
Durch diese Vererbung wird die Austauschbarkeit ermöglicht.

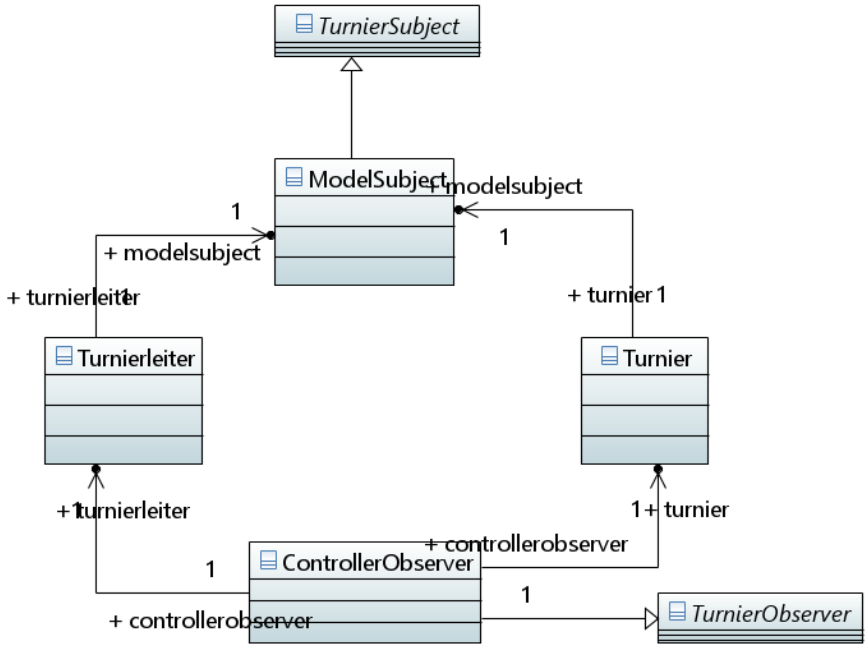


Abbildung 25: KDTurnierModell

3.5.4. Klassendiagramm KDTurnierView

SpielerAusgabe: Alle Methoden sind statisch und sind dafür da, damit was im Abhängigkeit von Spielerdaten für den Nutzer ausgegeben werden kann.

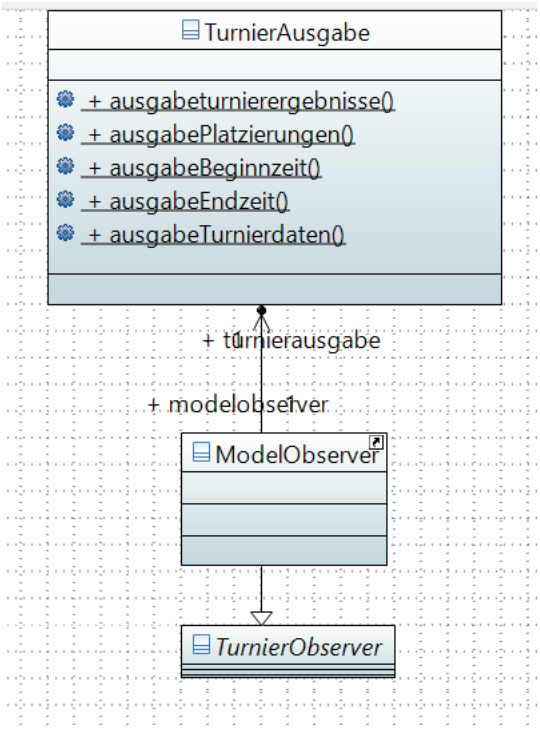


Abbildung 26: KDTurnierView

Am Anfang des FeinEntwurfs habe ich länger überlegt, ob ich einfach nur ein Modell, ein View und ein Controller anlege oder ob ich jeweils eins anlege.
Ich habe mich für die zweite Variante entschieden, um die beschreibenden Prinzipien (Seite 9) wie Abstraktion, Strukturierung, Bindung und Kopplung, Hieraeichierung und Lokalität beizubehalten. Außerdem lässt sich das Projekt so besser und einfacher entwicklen bzw. ändern, da man eine Änderung In einem Subsystem vornehmen kann unabhängig vom anderen Subsystem im anderen MVC. Beispielsweise kann man was im Modell Turnier ändern ganz unabhängig vom Modell Spiel und anderesum. Ich finde also mit dieser Entscheidung ist der Code anpassbar wie nie!

3.3. Architekturprinzipen

Folgende Architekturprinzipien wurden in dieser Hausarbeit genutzt:(Kapital 4, Seite 360)

- Prinzip der Einheitlichkeit: Entwurfsentscheidungen wurden im gesamten System berücksichtigt Und es wurde nur ein Entwurfmuster benutzt.
- Prinzip der Trennung von Zuständigkeiten: Durch das MVC hat jedes Subsystem seine eigene Aufgabe. Zusätzlich wurden Zuständigkeitsbereiche durch Pakete realisiert.
- Prinzip der Sichtbarkeit: Jedes Artefakt dieser Architektur wurde beschrieben und dokumentiert.
- Prinzip der Symmetrie: Die Klassen sind leicht und verständlich mit der Auswahl der Attributen und Methoden
- Prinzip der Einfachheit: Es wurde zwischen Grob- und Feinentwurf unterscheiden und die Architektur innerhalb MVC ist auch klar und in Unterpaketen unterteilt.

4. Implementierung

4.1. Papyrus Codeerzeugung

Der Quellcode wurde automatisch von Papyrus erzeugt und solle dann im Rahmed der Implementierung dienen.

Aufgrund meines Problems, welches ich am 17.02.2022 13:37 geschildert habe, war ich nicht mehr in der Lage das Projekt mit der Implementierung abzuschließen, denn ich mehrere Diagramme sowohl von der Analyse- als auch von der Entwurfphase nochmal machen musste. Außerdem hat es ab und zu auch viel mit den Klassenverträgen gedaurt, sodass ich die Fehlermeldungen behoben habe (mit 2 verschiedenen Eclipse Versionen 2020-12 und 2020-06)

Teil III.
Schluss

Dieses Softwaretechnik-Projekt hat mir ein gutes Verständnis in die Funktionsweise einer Softwareentwicklung gegeben und war auch sehr praxisnah.

Teil IV.
Literatur

[1] Prof. Dr. Wolf Zimmermann. Vorlesungsskript zum Modul Softwaretechnik

[2] Prof. Dr. Helmut Balzert. Lehrbuch der Softwretechnik: Entwurf, Implementierung, Installation und Betrieb. Spektrum Akademischer Verlag, 2011.

[3] eigene Notizen von der Übung bei Dr. Mandy Weißbach

[4] Logo der Universität Halle https://www.pr.uni-halle.de/download/logo/

Teil V.
Abbildungsverzeichnis

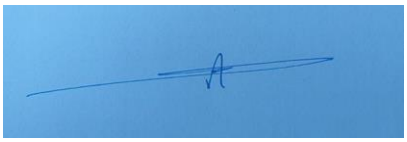
Abbildungsverzeichnis

1. AFModellierung 7
2. AFSpieler 8
3. AFTurnierleiter 9
4. AFAdministrator 10
5. AFSpielregeln 11
6. PDModellierung 12
7. KDSpielverwaltung 13
8. KDSpielerverwaltung 14
9. KDTurnierverwaltung 15
10. PDEntwurf 18
11. PDMuehleModel 19
12. PDMuehleView 20
13. PDMuehleController 21
14. KDMuehleMVC 22
15. KDSpielerObserver 23
16. KDSpielerController 24
17. KDSpielerModel 25
18. KDSpielerView 26
19. KDSpielObserver 27
20. KDSpielController 27
21. KDSpielModell 28
22. KDSpielView 28
23. KDTurnierObserver 29
24. KDTurnierController 29
25. KDTurnierModel 30
26. KDTurnierView 31

Teil VI.
Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig und ohne fremde Hilfe angefertigt habe. Textpassagen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Unterschrift



Halle, 22.02.2022

Teil VII.

Anhang:

Modul „Softwaretechnik“
Pflichtenheft (SWT WS 2021/22)

Ahmad Wattar

23.02.2022

1 Einleitung

Ab dem Wintersemester 21/22 soll im Rahmen des Moduls Softwaretechnik an der Martin-Luther-Universität Halle-Wittenberg ein Projekt durchgeführt werden. Die Studenten sollen dabei die in der Vorlesung vermittelten Grundfertigkeiten bei der Entwicklung von Software anwenden.

2 Ziel

Es soll eine Remote-Anwendung für das Spiel Mühle entwickelt werden.
Die entstehende Software soll in den folgenden Jahren erweitert werden. Dabei sollen dann z.B. weitere Spiele und andere Erweiterungen hinzugefügt werden. Die folgende Beschreibung bezieht sich dabei auf den Stand, welcher im WS 2021/22 entstehen soll.

3 Rahmenbedingungen

- Die Software muss gut dokumentiert in einem Papyrus-Modell nebst generiertem Code ausgeliefert werden
- Die Programmiersprache muss Java sein.
- Die erstellte Software muss auf den Rechnern der Softwaretechnik-Arbeitsgruppe lauffähig sein (Linux-Rechner).

4 Funktionale Anforderungen

4.0.1 Spiel Mühle

FA1.10: Spiel verwalten

FA1.10.1: Ein Spieler kann ein Spiel erstellen
FA1.10.2: Jedes Spiel hat folgende Daten

FA1.10.2.1: eine Startzeit
FA1.10.2.2: eine Endzeit
FA1.10.2.3: eine Spieldauer
FA1.10.2.4: einen Gewinner
FA1.10.2.5: einen Verlierer
FA1.10.2.6: die Anzahl der Züge pro Spieler

FA1.10.3:Jeder Spieler kann in jeder Zeit das Spiel aufgeben, dann hat er verloren
FA1.10.4: Der Administrator kann die Spielregeln modifizieren
FA1.10.5: Jeder Spieler hat die Möglichkeit entweder zuzweit zu spielen oder am Turnier teilzunehmen.

FA1.20: Turnier verwalten

FA1.20.1: Ein Turnier soll nur im KO-System ausgetragen werden. Allerdings sollen zu einem spätern Zeitpunkt andere Turniermodi hinzukommen. Die Auslosung soll automatisch sein aber es soll auch später möglich sein, unterschiedliche Methoden zur Aufstellung der Spiele bzw. Spieler zu wählen
FA1.20.2: Damit ein Turnier stattfinden kann, sollten mindestens 4 Spieler teilnehmen. Es gibt aber keine maximale Spieleranzahl geben
FA1.20.3: Der Administrator kann alle Turniere verwalten.
FA1.20.4: Jedes Turnier hat genau einen Turnierleiter und dieser Turnierleiter darf am selben Turnier nicht teilnehmen
FA1.20.4: Jeder Turnierleiter kann immer nur ein Turnier leiten, niemals mehrere gleichzeitig
FA1.20.5: Jedes Turnier beginnt und endet zu einem bestimmten Zeitpunkt und hat außerdem die Daten Spiele, Spieler, Turniermodus
FA1.20.6: Der Turnierleiter kann nur die für ihn eingerichteten Turniere bzw. die Spiele innerhalb der Turniere verwalten
FA1.20.7: Ein Spieler hat eine beste Plazierung, wenn er einen der ersten 4 Plätze erreichet
FA1.20.8: Wenn ein Spieler keinen Gegner hat, dann zieht er per Freilos in die nächste Runde
FA1.20.9: Der Turnierleiter kann einen Turnier starten, wenn die Turnier-Teilnehmer eindeutig sind, also einen Nutzernamen haben

FA1.30: Spielzug durchführen gemäß den Regeln des Spiels

FA1.30.1: Das Spielbrett besteht aus drei ineinander liegenden Quadraten, mit Verbindungslinien zwischen den Seitenmitten
FA1.30.2: Es gibt insgesamt im Spiel 18 Spielsteine. Ein Spieler hat 9 schwarze Steine und der andere hat 9 weiße Steine
FA1.30.3: die erste Phase im Spiel ist die Spielvorbereitung, in der jeder Spieler sich eine Farbe aussucht. In dieser Phase ist das Spielbrett nicht mit Steinen befüllt.
FA1.30.4: Spielziel ist erreicht wenn der Gegner weniger als 3 Steine hat oder der Gegner keinen Zug ausführen kann.
FA1.30.5: Ein Spieler konstruiert eine Mühle , wenn er 3 Steine in einer Geraden hat
FA1.30.6: Ein Spieler kann einen Stein vom Gegner wegnehmen, wenn der Spieler eine Mühle konstruiert hat. Dieser Stein darf nicht aus einer Mühle sein. Außer, dass der Gegner nur Mühlen hat.
FA1.30.7: Falls der Spieler mit einem Zug gleichzeitig zwei Mühlen bildet, dann darf er nur einen Stein vom Gegner wegnehmen.
FA1.30.8: in der ersten Spielphase (Setzphase) legen die Spieler abwechselnd ihre Steine auf einem Schnitt oder Eckpunkt des Spielbretts.
FA1.30.9: Der Spieler mit den weißen Steinen, fängt in der Setzphase an und legt einen den ersten Stein
FA1.30.10: Sofern eine Mühle bereits in der Setzphase entstanden wurde, dann darf der Spieler, der diese Mühle konstruierte, dem Gegner einen Stein wegnehmen
FA1.30.11:Nachdem die Spieler alle Spielsteine auf das Spielbrett gesetzt haben, endet die Setzphase und beginnt die zweite Phase im Spiel, die Zugphase
FA1.30.12: in der Zugphase darf der Spieler einen seiner Steine auf einen benachbarten

Schnitt- oder Eckpunkt ziehen aber dieser Punkt muss frei sein, also nicht von einem anderen Stein belegt ist.
FA1.30.13:
Ein Spieler darf nicht in zwei Zügen hintereinander die selbe Mühle bilden. Also einen Stein aus einer Mühle herausnehmen und im nächsten Zug wieder hinrücken ist nicht erlaubt. Eine Ausnahme ist aber eine Zwickmühle
FA1.30.14:wenn ein Spieler nur 3 Steine hat, dann fängt die nächste Phase an, die Sprungphase
FA1.30.15: in der Sprungphase kann der Spieler seinen Stein vom einem Schnitt- bzw. Eckpunkt zu einem beliebigen Schnitt- bzw. Eckpunkt bewegen
FA1.30.16:Wenn ein Spieler weniger als 3 Steine hat, dann ist dieser Spieler verloren und der andere gewinnt.
FA1.30.17: Wenn von keinem der Spieler nach 20 Zügen eine Mühle gebildet wurde, oder dreimal in Folge die gleiche Stellung der Spielsteine erreicht wird, endet das Spiel mit Unentschieden.
FA1.30.18: Ein Spieler hat verloren, wenn er die angegegbene Zeit überschreitet oder die Verbindung abbricht.

FA1.40: Spielerverwaltung

FA1.40.1:Jeder kann sich registrieren um einen Spieler zu erstellen.
FA1.40.2: Jeder Spieler hat folgende Daten

FA1.40.2.1: einen Namen
FA1.40.2.1: einen Nutzername
FA1.40.2.1: ein Geburtsdatum
FA1.40.2.1: ein Passwort
FA1.40.2.1: ein Spielscore
FA1.40.2.1: eine Anschrift

FA1.40.3: Der Administrator kann alle Spieler verwalten, d.h. alle Daten ändern bis auf das Spielscore
FA1.40.4: Der Spieler kann seinen Namen oder Geburtsdatum ändern, Anschrift oder Mail aber nicht
FA1.40.5 Der Spieler kann seine persönliche Daten abrufen
FA1.40.6 Jeder Spieler kann seinen Account löschen
FA1.40.7 Jeder Spieler hat in seinem Account die Anzahl der gespielten Spiele, der gewonnen Spiele, Anzahl der gespielten Turniere und Anzahl der gewonnen Turniere bzw. beste Platzierung

5 Nichtfunktionale Anforderungen

LNA1000:Die Spielerdaten sollen geschützt sein.
LNA2000:Qualitätsanforderungen sollen erfüllt werden.
LNA3000:Nur ein Spiel zu einem Zeitpunkt
LNA4000:Nur ein Turnier zu einem Zeitpunkt
LNA5000:Wenn ein Spiel oder ein Turnier stattfindet, kann kein Spieler sich zum Spiel anmelden oder ein neues Turnier gestartet werden
LNA6000:Administatortätigkeiten können nicht während eines Spiels oder Turnier durchgeführt werden

6 Nutzerschnittstelle

Die Interaktion zwischen dem Nutzer und dem Spiel soll über die Konsole realisiert werden und dazu wird keine GUI benötigt.

7 Qualitätsanforderung

LQA1000: Die Software muss gut dokumentiert sein.

Systemqualität	sehr gut	gut	normal	irrelevant
Funktional				
Angemessenheit		X		
Genauigkeit		X		
Interoperabilität	X			
Sicherheit	X			
Konformität			X	
Zuverlässigkeit				
Reife	X			
Fehlertoleranz		X		
Wiederherstellbarkeit	X			
Konformität			X	
Benutzbarkeit				
Verständlichkeit	X			
Erlernbarkeit	X			
Bedienbarkeit		X		
Attraktivität		X		
Konformität			X	
Effizienz				
Zeitverhalten	X			
Verbrauchsverhalten			X	
Konformität			X	
Wartbarkeit				
Analysierbarkeit	X			
Änderbarkeit		X		
Stabilität	X			
Testbarkeit			X	
Konformität			X	
Portabilität				
Anpassbarkeit		X		
Installierbarkeit	X			
Koexistenz		X		
Austauschbarkeit			X	
Konformität		X		

8 Abnahmetests

A10 Gültiges Abnahmeszenario:
1. registriere Eva passwort abc
2. OK . Eva ist weiss .
3. registriere Max passwort xyz
4. OK . Weiss ist dran .
5. Setze Stein auf 1 , N
6. OK . Schwarz ist dran .
7. Setze Stein auf 1 , SW
8. OK . Weiss ist dran .
9. Zeige Spielfeld .
10. (1 , N): weiss
11. (1 , SW): schwarz
12. alle anderen Punkte sind nicht belegt bzw. ”leer”

A20 Ungültiges Abnahmeszenario:
1. Absturz

9 Glossar

9.1 Begriffe

- **Spieler**:
  Ist Eine Entität, die Spiele spielen kann oder Turniere startet und verwaltet

- **Turnierleiter**:
  Jeder Spieler kann zu einem Turnierleiter werden, indem er ein Turnier startet. Dieser Turnierleiter kann alle Spiele die zu diesem Turnier gehören verwalten

- **Administrator**:
  Kann alle Turnier, Spiele und die Spielerdaten verwalten, also ändern oder löschen, bis auf das Spielscore, um Manipulation zu vermeiden

- **Turnier**:
  Ein Turnier ist eine Sammlung von mehrern Spielen und mindestens 4 Spieler

- **KO-System**:
  K.o. ist eine Abkürzung für das Wort „Knockout“ und bedeutet wer das Spiel verliert, scheidet aus

- **Spielscore**:
  Das Spielscore eines Spielers besteht aus Anzahl der gespielten Spiele und Turniere, Anzahl der gewonnen Spiele und Turniere und Anzahl der verlorenen Spiele und Turniere

- **beste Platzierungen**:
  Die ersten 4 Plätze in einem Turnier sind die besten Platzierungen

- **Mühle**:
  Eine Mühle entsteht, sobald drei Spielsteine einer Farbe auf einer Geraden nebeneinander liegen. Dies gilt sowohl für vertikale, als auch für horizontale Geraden. Wenn ein Spieler eine Mühle bildet, darf er einen Spielstein des Gegners vom Spielbrett nehmen

- **Zwickmühle**:
  Stellung der Steine im Mühlespiel, bei der der Spieler durch Hin- und Herschieben eines Steines jeweils eine neue Mühle hat.

- **Spielphasen**: Das Spiel besteht aus 3 verschiedenen Phasen, in denen einige Regeln sich unterscheiden

- **Setzphase**:
  Ist die erste Phase im Spiel und ist dazu da die Spielsteine abwechselnd auf dem Spielbrett zu platzieren

- **Zugphase**:
  In dieser Phase können die Spieler ihre Spielsteine auf einem benachbarten Punkt bewegen, wenn dieser frei ist

- **Sprungphase**:
  Ist die letzte Phase im Spiel und fängt an sobald ein Spieler nur 3 Steine auf dem Spielbrett hat. Hier kann der Spieler seinen Stein auf einem beliebigen Punkt springen

9.2 Spieldarstellung

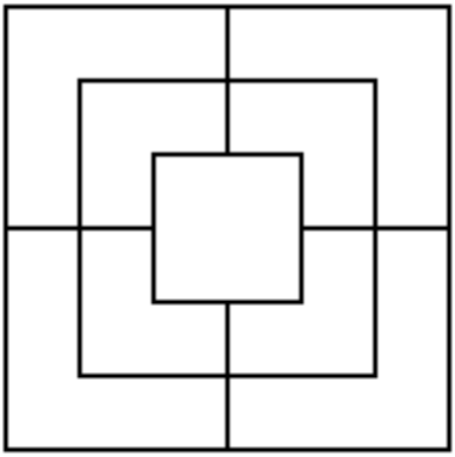


Abbildung 1: Spielbrett
(Quelle: Wiki, Studip)

9.2.1 Legende

Die Beschreibung der Knoten erfolgt von oben links (außen) bis unten rechts (außen) für alle Eck- und Schnittpunkte, dabei sind 1 := innen, 2 := mitte, 3:= außen:

3.NW	3.N	3.NO			
2.NW	2.N	2.NO			
1.NW	1.N	1.NO			
3.W	2.W	1.W	1.O	2.O	3.O
1.SW	1.S	1.SO			
2.SW	2.S	2.SO			
3.SW	3.S	3.SO			